

ROS2

DAY 4 – Manual 1

ROS2 Services – Fundamentals & Setup

Complete Beginner-Friendly

Objective

By the end of Manual 1, you will be able to:

1. Understand the concept of **ROS2 services**.
 2. Know the difference between **Topics and Services**.
 3. Create a ROS2 Python package for services.
 4. Create `.srv` files and understand their structure.
 5. Update `package.xml` and `setup.py` for services.
 6. Build the package and generate Python service interfaces.
-

Step 0 – Pre-Requisites

Before starting:

- ROS2 installed (Foxy, Humble, Rolling)
 - Python 3 installed
 - A ROS2 workspace (`ros2_ws`) created
 - Basic understanding of ROS2 nodes and topics (from Day 1 & 2)
-

Step 1 – Understand ROS2 Services

1.1 What is a Service?

- A **service** is a mechanism in ROS2 that allows a **client node** to send a **request** to a **server node** and wait for a **response**.
- Unlike topics (which continuously stream data), services are **synchronous**: the client sends a request and waits for the server's reply.

Example:

```
CLIENT → "What is 4 + 5?"  
SERVER → "It is 9."
```

1.2 Service vs Topic

Feature	Topic	Service
Communication	One-way stream	Two-way request/response
Frequency	Continuous	Only when requested
Example	Sensor data	Commands, calculations

1.3 Anatomy of a .srv File

- .srv files define the **request and response data structure**.
- Structure:

```
# Request (top)  
<type> <field_name>  
---  
# Response (bottom)  
<type> <field_name>
```

Example: StringCommand.srv

```
string command  
---  
string response
```

- command → what client sends
 - response → what server replies
-

Step 2 – Create ROS2 Package for Services

1. Open a terminal.
2. Navigate to your ROS2 workspace source folder:

```
cd ~/ros2_ws/src
```

3. Create a Python package named day4_services:

```
ros2 pkg create --build-type ament_python day4_services
```

4. Navigate into the package folder:

```
cd day4_services
```

Explanation:

- `ros2 pkg create` → generates a standard Python ROS2 package
 - `--build-type ament_python` → tells ROS2 it's a Python package
-

Step 3 – Create `.srv` Folder and File

1. Create a folder to store service definitions:

```
mkdir -p srv
```

2. Create the service file `StringCommand.srv`:

```
nano srv/StringCommand.srv
```

3. Add content:

```
string command
---
string response
```

Explanation:

- **Top section:** request fields (`command`)
 - **Bottom section:** response fields (`response`)
 - Save and exit (`CTRL+O`, `ENTER`, `CTRL+X`)
-

Step 4 – Update Package Files

4.1 Update `package.xml`

Add dependencies for service generation:

```
<build_depend>roscpp</build_depend>
<exec_depend>roscpp</exec_depend>
```

Explanation:

These dependencies allow ROS2 to generate Python code for your `.srv` file.

4.2 Update `setup.py`

1. Include the `.srv` file in `data_files`:

```
data_files=[  
    ('share/day4_services/srv', ['srv/StringCommand.srv']),  
],
```

2. Add entry points for the server and client nodes (you'll create these later in Manual 2):

```
entry_points={  
    'console_scripts': [  
        'string_server = day4_services.string_server:main',  
        'string_client = day4_services.string_client:main',  
    ],  
},
```

Explanation:

- `data_files` → ensures `.srv` files are installed with the package
- `entry_points` → registers Python scripts as ROS2 executable nodes

Step 5 – Build the Package

1. Navigate to your workspace root:

```
cd ~/ros2_ws
```

2. Build using `colcon`:

```
colcon build
```

3. Source the workspace:

```
source install/setup.bash
```

Explanation:

- This generates the Python code for your `.srv` file
 - Ensures ROS2 recognizes your new package
 - Workspace is ready for server/client implementation
-

Step 6 – Verify the Setup

1. List all packages to confirm:

```
ros2 pkg list | grep day4_services
```

- You should see `day4_services` listed.

2. List services (none exist yet):

```
ros2 service list
```

- Empty now because we haven't created server nodes yet

Explanation:

- Setup is complete and ready for coding
 - Manual 2 will focus on **writing server and client nodes, line by line**
-

Step 7 – Concepts Recap

- **Services** are request/response based, **synchronous** communication
 - **Server node** waits for requests
 - **Client node** sends requests and waits for responses
 - `.srv` file defines the data structure for request & response
 - Setup steps include creating package, `.srv` file, updating `package.xml` & `setup.py`, and building
-

✓ **Manual 1 Complete!**

Next step (Manual 2) will cover **writing the server and client code**, explaining **each line**, and **running & testing the service** in ROS2.
